

- קובי חזון
- יצחק הראל
- רועי נאור

סיכום פרויקט -

סדנא בתכנון תנועה לרובוטים

מטרת הפרויקט:

לבצע תנועה אוטונומית של רחפן במבנה סגור המורכבת ממסלול בין תגים הממוקמים על קירות המבנה, תוך כדי שילוב כלים, מערכות ואלגוריתמים מודרניים לעבודה עם רובוטים.

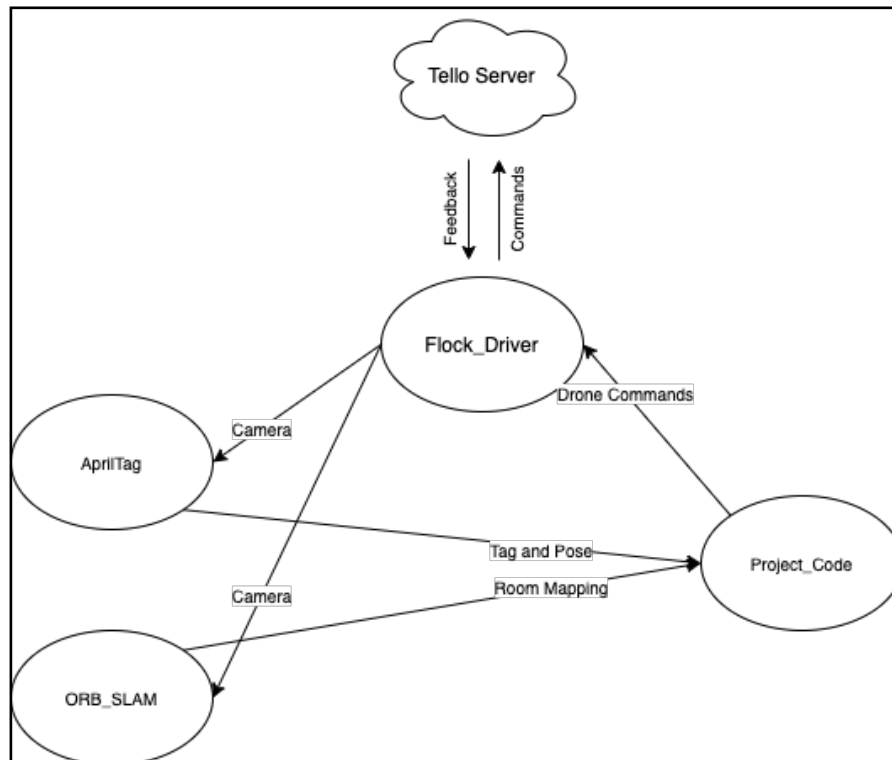
תהליך הלמידה של הקבוצה:

כאשר התחלנו את הפרויקט, היינו חסרי כל ידע וניסיון בנושאים שאמרו לנו שיהיו מאוד חשובים ומעין אבני יסוד לשאר תהליך הלמידה והעבודה על הפרויקט. נושא עיקרי שכזה, הינו ROS שכן ידענו שכל הפיתוח והקוד שלנו יפעל תחת התהליך הראשי של ROS ובכך כל חלק בקוד ינוהל בפשטות כ-Node ב-ROS. לפיכך, החלטנו שנקדיש על ההתחלה חודשיים על מנת ללמוד את אותם אבני יסוד שעליהם ייבנה הפרויקט שלנו. לקחנו קורס אינטרנטי ב-ROS ולאחר שסיימנו אותו בזריזות, עברנו לשיעורים בנושאים מתקדמים יותר ב-ROS, שהועברו בלייב ובחינם בכל ערב במשך כ-3 שעות. לאחר שהרגשנו נוחים עם ROS, התקנו את סביבת העבודה שלנו וניסינו לעשות כמה תרגילי מבוא בנושא. השתמשנו במערכת הפעלה Ubuntu 18.04 עליה התקנו את ROS Melodic. לאחר מכן למדנו כיצד להשתמש בספריות שסיפקו לנו את זיהוי התגים, מיפוי החדר ואיכון הרחפן בסביבה, ספריות ה-AprilTag וה-ORB_SLAM. בהמשך לשלב האחרון והתקנת הספריות בהצלחה בסביבת העבודה שלנו, עברנו להתקנת חבילה שמקלה על השליטה שלנו ברחפן, בכך שהיא מספקת לנו אובייקט המאותחל ספציפית לרחפן שלנו (על ידי התחברות אליו ישירות ב-wifi של המחשב) ובאמצעות מתודות שונות אנו יכולים לשלוט ישירות בתנועות הרחפן מאשר שידנית נבנה ונשלח פקטות לרחפן עם המהירויות שאנו רוצים לתת לו בכל צירי התנועה שלו (ועוד כל מיני פקודות שונות). עם החבילה הזאת הייתה לנו בעיה בהתקנתה ושילובה עם ה-ROS ולכן נדרשנו לבצע מספר לא קטן של שינויים בקוד עד שהצלחנו לגרום לה לעבוד בצורה מספקת. כעת יצאנו לדרך עם הפרויקט.

מרכיבי הפרויקט הסופי:

- למידה ושימוש ב-ROS ומרכיביו השונים.
- היכרות כללית עם אלגוריתמים לזיהוי עצמים בתמונה והעמקה בספריית AprilTag, עבור זיהוי תגים, ושימוש בהם לאיכון יחסי לתג.
- היכרות כללית עם אלגוריתמים למיפוי הסביבה וספציפית באלגוריתם ה-ORB_SLAM, על מנת למפות את החדר, להימנע מהתנגשויות ולבצע תנועה ללא תג.
- התממשקות עם הרחפן באמצעות אות ה-WiFi שהוא מפיץ ושליטה בו כמתואר ב-SDK הרשמי.
- תכנון תנועה לעבר תגים ולחיפוש תגים המתחשבת במגבלות הרחפן ובעיותיו.
- שיפורים אחרונים לנוחות: יצירת GUI לשליטה נוחה במרכיבי הפרויקט וקבלת מידע וכן כתיבת סקריפט המשייך פקודות שונות להגעה לתגים שונים (למשל ביצוע היפוך לכיוונים שונים).

תרשים ROS של הפרויקט:



עבודה עם AprilTag:

AprilTag הינה ספרייה הממשת אלגוריתם המספק איכון מלא של מצלמה במידה ותג מתאים נמצא בתמונה שלה. על מנת שתג יהיה מתאים, עליו להתאים לסטנדרטים עבור ברקוד של AprilTag (שיהיה בגודל מסוים ומסביב לברקוד יהיה רקע לבן). מעבר לכך, על האלגוריתם לקבל כקלט מידע על המצלמה שמעבירה לו תמונות (זווית הראייה של המצלמה לגובה ולרוחב) וכן לדעת את גודל התג. בהינתן מידע זה, הספרייה מספקת לנו את מיקום המצלמה יחסית לתג במרחב ואת האוריאנטציה שלה.

אנו משתמשים בפלט האלגוריתם על מנת לתכנן את תנועתנו לעבר התג ובסופו של דבר להגיע אליו. כמו כן, הוספנו פונקציונאליות של קידוד לתג פקודה מסוימת עבור הרחפן, על מנת שההדגמה תהיה מהנה ודינאמית. פקודות אפשריות הן למשל להגיד לרחפן לבצע היפוך בכיוון מסוים, לקחת תמונה או להסתובב במקום.

עבודה עם ORB_SLAM:

ORB_SLAM הינו אלגוריתם לאיכון ומיפוי בזמן אמת שבו השתמשנו על מנת לנסות ולהקטין את כמות ההתנגשויות עם הקירות ולייעל את התנועה של הרחפן כאשר אין בתמונה שלו אף תג שלא ראה בעבר (כי אם זה תג חדש, הוא יכול להיעזר בספרייה ה-AprilTag על מנת להגיע אליו).

האלגוריתם עובד על ידי כך שהוא מחפש נקודות בתמונה שהן יחסית משמעותיות (כלומר, לא נקודות רנדומליות על קיר לבן. אלא פינה של שולחן/קיר/חלון) ובתמונה הבאה שהוא מקבל הוא ינסה לקשר בין הנקודות הישנות למיקום החדש שלהן בתמונה החדשה. לאחר מספר פעמים שהוא חוזר על התהליך הזה, הוא יכול לייצר "ענן נקודות" (נקודות תלת מימדיות בחדר הנסרק) והמיקום של המצלמה בתוכה. אל "ענן הנקודות" (באנגלית - Point Cloud) הוא יוכל להוסיף נקודות במידה ותהיה לו אינפורמציה נוספת או לבטל אותו במידה ואיבד את הנקודות שסרק.

באמצעות נקודות אלה אנחנו יכולים להשתמש בכלים שבאים עם ROS על מנת ליצור משטחים (Triangle Mesh) ולנסות להחליק אותם כדי להתקרב כמה שיותר למבנה האמיתי של החדר. באמצעות מיפוי זה של החדר אנו יכולים לנסות ולא להתנגש בקיר (למרות שזה מאוד מוגבל בגלל סטייה גדולה בתנועת הרחפן וחוסר יציבות שלו) וכן לנסות לייעל את החיפוש של תג נוסף, על ידי ניסיון להגיע למרכז החדר או קירוב שלו ושם סיכויי גדול יותר שנראה תג נוסף על ידי סיבוב במקום.

עבודה עם הרחפן DJI Tello:

אנו יוצרים חיבור אל הרחפן על ידי אות ה-WiFi שהוא מפיץ וכפי שמתואר ב-SDK של הרחפן, אנו שולטים בו על ידי שליחת פקטות UDP עם מחרוזת עבור הפקודה הרצויה לפורט מסוים. כמו כן, הרחפן שולח לנו בתדירות גבוהה את התמונות מהמצלמה שלו וכן מספר פרמטרים המתארים את המצב שלו - אחוז הסוללה, האם הוא עף, האם יש כשל מסוים, טמפרטורה וגובה הרחפן. אנו שולטים בתנועת הרחפן על ידי שליחת 4 מהירויות התואמות לדרגות החופש של הרחפן - תנועה במרחב וכן סיבוב סביב עצמו.

מנקודת מבטנו, יש לרחפן מספר יתרונות משמעותיים. הראשון הוא הפשטות שבה ניתן לשלוט עליו מרחוק ובכך לתכנת אותו לבצע באופן אוטונומי דברים שונים. כמו כן, המחיר הזול של הרחפן (100 דולרים) הופכים אותו לנגיש מאוד לפרויקטים קטנים. יתרה מכך, יש לו מצלמה סבירה שמכוונת קדימה וחיישן TOF שמכוון מטה והם מאוד חשובים בעבודה עם רחפנים והם מאפשרים למישהו שחדש שעולם הרחפנים לקבל תמונה יחסית מלאה של מה שאפשר לעשות עם רחפנים פשוטים ללא פשרות.

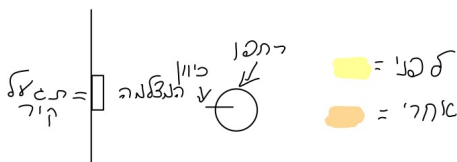
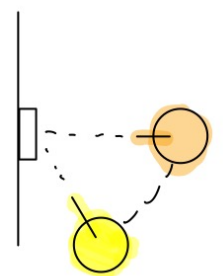
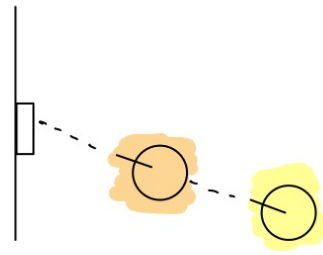
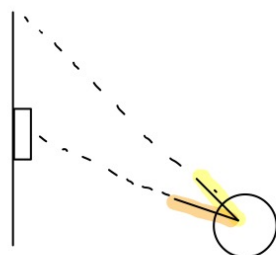
עם זאת, יש לו גם חסרונות. הבולט מביניהם הוא מערכת הייצוב של הרחפן. הרחפן מנסה לייצב את עצמו (גם כאשר שולטים עליו מרחוק, ואין אפשרות לכבות את מערכת הייצוב) על ידי שימוש במצלמה מכוונת מטה (שלצדדי לא נגישה למפתחים) והחיישן לייזר. באמצעות שני חיישנים אלה הוא מנסה לראות האם הוא זז למרות שאמור להישאר במקום ולתקן בכך את מיקומו. הבעיה היא שעל משטחים שהם "חסרי מרקם" או מאוד אחידים (כמו רצפת שיש אחידה וחלקה או דף לבן על הרצפה), הוא לא אמין ויוצא מצב אירוני שמערכת הייצוב מוציאה את הרחפן מייצוב ומזיזה אותו. כמו כן, קיימת סטייה משמעותית בכל תנועות הרחפן, בעיקר מבחינת הכיוון והזמן שלוקח לרחפן לעצור/לשנות את מהירותו.

תכנון תנועת הרחפן:

נתאר את תנועת הרחפן בשני מצבים: ללא מטרה ועם מטרה (מטרה=תג במקרה שלנו). בעבור תנועה ללא מטרה, אנו מבצעים תנועה של עלייה עד למרחק של עשרים סנטימטרים מהתקרה וירידה עד למרחק של חצי מטר מהרצפה, לאחר מכן נסתובב חלק מסיבוב כך שהמצלמה כעת רואה זווית חדשה של החדר שלא ראתה בעבר וחוזרים חלילה. אנו חוזרים על הפעולה המתוארת עד שסיימנו סיבוב שלם, אם במהלך תנועה זו ראינו תג, כמובן שנפעל בהתאם ונגיע אליו.

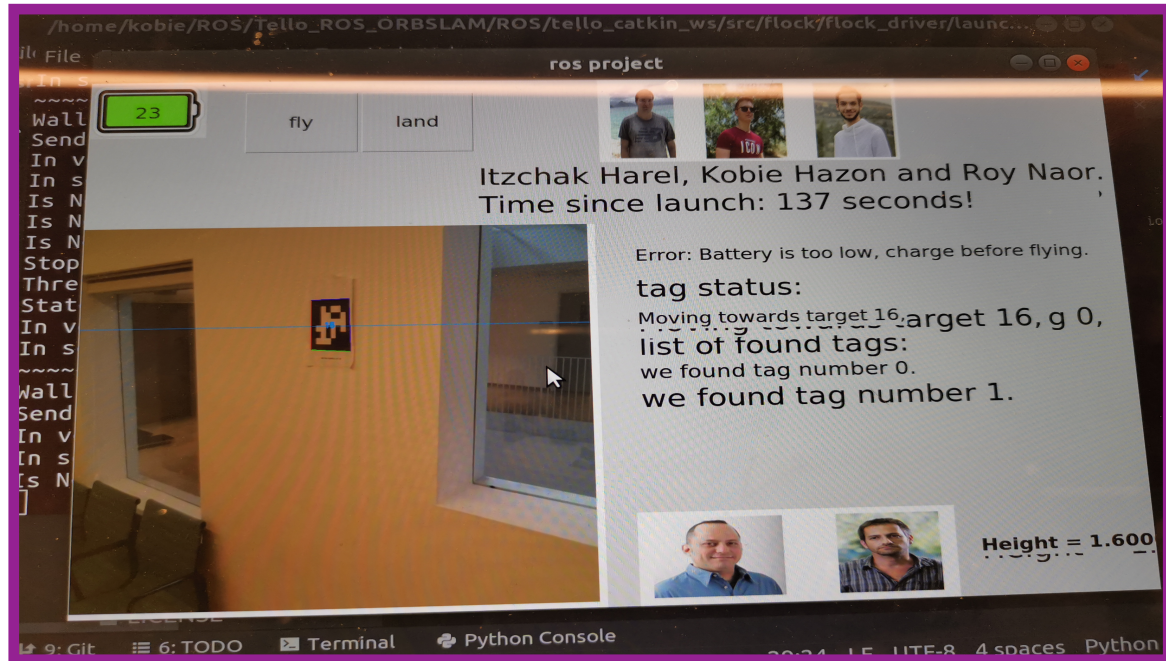
בעבור תנועה עם מטרה, בעולם מושלם יכולנו לחשב את הזווית שהרחפן צריך לזוז כדי להגיע אל המטרה, לתת לו מהירות קבועה ולחשב את זמן התנועה ככה שנעצור אותו כשהוא יגיע למטרה. לצערנו, זה לא ישים בשל המגבלות של הרחפן שתיארתי בחלק האחרון של "עבודה עם הרחפן DJI Tello". לכן פיצינו על כך על ידי פיצול התנועה לעבר התג לשלוש תנועות שונות (כל אחת מהן מורכבת יותר אך נפרט על כך בהמשך, בחלק של האתגרים). הסוגים הינם:

1. תנועת רוטציה לעבר המטרה - כאשר המטרה אינה במרכז פריים התמונה ונמצאת קרוב מדי לקצוות התמונה, נסובב את הרחפן סביב עצמו (ותוך כדי נעלה ונרד בגובה כדי להגיע לגובה התג) על מנת שהתג יהיה במרכז התמונה (איור שמאלי).
2. התקדמות לעבר התג - חישוב הכיוון שנצטרך לנוע בו על מנת להתקדם בקו ישר לעבר התג ותנועה בכיוון זה (איור אמצעי).
3. תנועה סיבובית כדי להתיישר מול הקיר - אם במהלך תנועה מסוימת אנו בזווית חדה מדי יחסית לקיר, נעשה עם הרחפן חלק מסיבוב על מנת שנהיה מאונכים לקיר עם הרחפן. כך אנו מורידים את הסיכון שנתנגש עם הקיר שעליו מודבק התג, במקרה ואנו ניגשה לתג מהצד (איור).



תוספות ושיפורים נוספים:

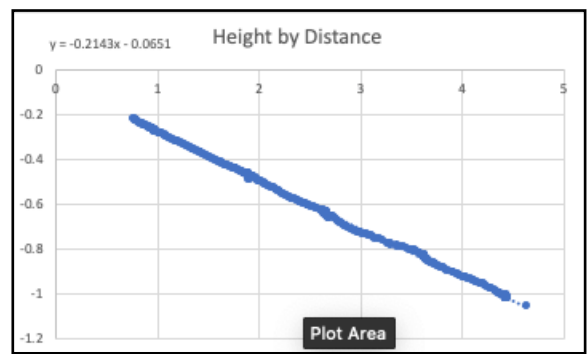
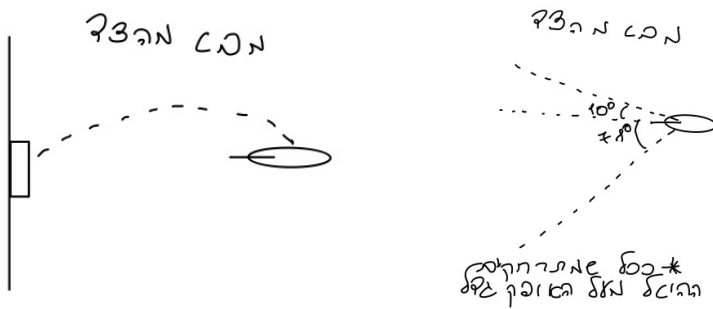
- i. **ממשק גרפי** - יצרנו ממשק גרפי שדרכו ניתן להתחיל ולהפסיק את פעולת הרחפן וכן לקבל פידבק על פעולות הרחפן והדברים שהוא עשה כבר. בממשק אנו יכולים לראות את אחוז הסוללה הנוכחי, את גובה הרחפן, הטמפרטורה שלו, האם יש שגיאות מסוימות וכן מידע לגבי המטרות בהם ביקר. המידע על המטרות כולל את הסטטוס הנוכחי של הרחפן (מסתובב לעבר מטר/מתקדם לעבר מטר/עושה תנועה סיבובית למטר/מחפש מטר חדשה/איבד מטר/הגיע בהצלחה למטר), וכן רשימה של המספרים המזהים של כל התגים אליהם הרחפן הגיע בהצלחה.



- ii. **סקריפט לשיוך פקודות לתגים** - על מנת שהפעלת הקוד תהיה אינטרקטיבית יותר, ניתן לשייך פקודה לכל תג והפקודה תבוצע כאשר הרחפן הגיע לתג. הפקודות כוללות "פליפים" (היפוך של הרחפן לכיוונים שונים: קדימה, אחורה וצדדים), לעשות סיבוב ולעלות מעלה מטה.
- iii. **מערכת למניעת התנגשות בתקרה** - במידה והרחפן נוגע בתקרה, הוא נצמד אליה והדרך למנוע את התרסקותו זה להנחית אותו. לכן אנו מנסים לשמור מרחק ביטחון מהתקרה אך אין דרך ישירה לדעת את גובה התקרה. אנו יודעים את מרחק הרחפן מהרצפה ולכן השתמשנו בזה ובקלט מהמשתמש לגבי גובה התקרה לשם כך. יש גובה ברירת מחדל של 2.4 מטר (גובה חדר ממוצע).

אתגרים בתכנון תנועת הרחפן:

- בעיית גובה הרחפן** - כאמור, ספריית ה-AprilTag מספקת לנו איכון של הרחפן באופן יחסי לתג, ואנו משתמשים באיכון זה על מנת להגיע לתג. אך מהר מאוד ראינו כי יש בעיה עם הגובה שמוחזר לנו והיא שיש שגיאה גדולה בגובה הרחפן התלוייה במרחק הרחפן מהתג. ככל שאנו מתרחקים, לגובה שגיאה כלפי מטה ולכן נפצה על כך ונעשה תנועה קשתית (ראה איור שמאלי).
- ראשית ניסינו לתקן בעיה זו על ידי חישוב טריגונומטרי שכן אנו מאמין שהשגיאה נובעת מהזווית שבה המצלמה ממוקמת על הרחפן (איור אמצעי), אך זה סיפק שיפור חלקי ולא מספק ולכן נגשנו לבעיה מזווית אחרת.
- הבחנו כי במידה והרחפן בגובה התג (גובה 0) וכן הם צמודים (מרחק 0), אז אין שגיאה כלל. ואילו ככל שאנו מתרחקים, אנו רואים שהשגיאה בגובה עולה ביחס קבוע. לכן מצוידים בהשערה זו, הכנו סקריפט שפולט לקובץ זוגות של מרחק וגובה והרצנו אותו כאשר הרחפן על מסילה שבאותו הגובה עם התג (גובה 0), כך שהגובה הנמדד ייצג רק את השגיאה. עם הזוגות הללו בנינו גרף (איור ימני) וראינו שאכן השערתנו נכונה ושיחס השגיאה הינו -0.2143, כלומר על כל מטר מרחק, נקבל גובה שהוא נמוך ב-21 סנטימטרים מהגובה האמיתי. כעת על ידי תיקון השגיאה באמצעות היחס, הגענו לתוצאות מאוד טובות.



בעיית זמן עצירת הרחפן - בשל מערכת הייצוב הפנימית של הרחפן והסטייות בתנועתו, כאשר נתנו לרחפן פקודה לעצור, המרחק שהיה עובר עד שעוצר (כי בכל רחפן העצירה לא מיידית) אינו קבוע אפילו כאשר לפני העצירה היה לרחפן את אותה המהירות. לכן היינו צריכים לקחת טווח ביטחון שתלוי במהירות הרחפן (כי בכל זאת מרחק העצירה תלוי במהירות לפני כן) כאשר אנו מתקדמים לעבר התג. עשינו זאת על ידי שבכל פקודה של התקדמות לעבר התג ננסה להקטין את המרחק מהתג בחצי, כאשר המהירות שבה נתקדם תלויה במרחק מהתג (יצרנו נוסחא למהירות כתלות במרחק שנותנת לנו מהירות מתאימה וטובה). כעת כשיש לנו ביד את המהירות והמרחק שאנו רוצים לעבור, חישבנו את הזמן שאנו צריכים לתת לפקודה הזאת (לפני שנעצור את הרחפן), ובזמן הזה אנו גם מחשבים את המהירות שצריך לתת לרחפן בצירי המרחב האחרים (לעלות/לרדת וללכת לצדדים - תוך כדי תיקון שגיאת הגובה) על מנת שבתום התנועה הוא יאפס את הצירים האחרים. כאשר אנו מספיק קרובים לתג (קבענו בקוד 2 מטר), נעבור למהירות קבועה ואיטית (שכן אנו צריכים תנועה מדויקת כשאנו במרחק שכזה מהקיר).

בעיית סטיית הרחפן והתערבות מערכת הייצוב - כאשר ננעלנו על מטרה, ניסינו למנע את האפשרות שמרכיבים אלה יגרמו לכך שנאבד את מטרתנו על ידי פיצול התנועה לשלוש תנועות שונות: סיבוב סביב עצמו, התקדמות למטרה ותנועה סיבובית להתיישרות מול הקיר (אותם תיארונו בחלק של תכנון תנועת הרחפן).

דיוק החיישנים ונתוני האיכון בו אנו משתמשים - כמצופה כאשר עובדים במרחב הפיזי, יכולת הדיוק של החיישנים ומכאן שגם דיוק האיכון של הרחפן הינה סופית וכמובן שדיוק תנועת הרחפן עצמו לא מושלם, ולכן בכל בדיקה של פרמטר איכון או זמן, אנו לא מנסים להבטיח שוויון מוחלט, אלא שוויון עד כדי אפסילון מסוים. מכאן שיצרנו אפסילונים שונים שמהווים את מידת הדיוק המקובלת על ידינו בעבור קואורדינטות שונות של האיכון.

אתגרים רבים נוספים שלא יכולנו להתמודד עמם: מדידת גובה התקרה על מנת שלא להתנגש בה (מבלי למדוד ידנית את הגובה לפני ההפעלה), יציבות אות ה-WiFi - בשל האות החלש שמשדר הרחפן, אם יש הרבה מחשבים מסביב לרחפן (אפילו מעל לחמש), הרחפן לא מגיב מספיק מהר לפקודות ולכן לא עוצר כשאמור ורוב הסיכויים שיתנגש (כמו כשהדגמנו בקומה רביעית את הפרויקט), זמן הסוללה - הרחפן יכול לטוס רק ל-12 דקות ברצף ואם יש לו פחות מ-30 אחוז סוללה הוא לא יסכים להמריא (ועם פחות מ-50 אחוז לא יבצע טריקים כמו היפוכים), מערכת הייצוב וסטיות בעוצמה משתנה - התמודדנו עמם אבל אי אפשר באופן מוחלט, ולכן זה עדיין יכול לגרום להתנגשויות/איבודי מטרה לא צפויים.

